Web Application Penetration Testing Report

For: Nothing Technology

Target Application: https://ae.nothing.tech/

Testing Type: Web Application Penetration Testing (Black-Box)

Prepared by: Sagar V - Security Analyst

Date: Week 1 Assessment

Executive Summary

This report presents the results of a **Black-Box Web Application Penetration Test** conducted against the **Nothing UAE website** (**https://ae.nothing.tech/**).

The purpose of this assessment was to identify potential security vulnerabilities that could be exploited by attackers. Testing was conducted without internal knowledge of the system architecture, source code, or infrastructure configuration, simulating a real-world external threat.

The assessment followed industry-recognized security testing standards, including:

- OWASP Web Security Testing Guide (WSTG)
- OWASP Top 10
- PTES (Penetration Testing Execution Standard)

During the engagement, the application was tested for common web application vulnerabilities, such as:

- Authentication weaknesses
- Input validation issues
- Misconfigurations
- Access control flaws
- Injection vulnerabilities

- Information disclosure

The findings identified during this assessment are documented in this report, along with their associated **risk levels, technical details, proof of concept, and remediation recommendations**.

The goal of this report is to help improve the application's security posture and reduce the risk of exploitation.

1.1 Project Objectives

The main objectives of this penetration testing engagement were to:

- Identify security vulnerabilities in the web application.
- Assess the application against **OWASP WSTG security controls**.
- Evaluate the application's resistance to common web attacks.
- Determine potential security risks that could impact the confidentiality, integrity, and availability of the system.
- Provide actionable remediation recommendations to improve security.

1.2 Scope & Timeframe

The scope of this project was limited to **external testing of the Nothing UAE website**.

Testing was conducted from an **external attacker perspective**, without access to internal documentation or source code.

Testing was performed during **Week 1 of the WAPT assessment period**.

1.2.1 In-Scope Asset

Asset	Description
https://ae.nothing.tech/	Public web application of Nothing UAE

1.2.2 Out-of-Scope Assets

The following assets were **excluded from testing**:

- https://*.nothing.tech

Subdomains, internal services, APIs, and infrastructure outside the defined scope were **not tested during this assessment**.

1.3 Summary of Findings

Security testing was performed according to the **OWASP Web Security Testing Guide (WSTG)** methodology.

The following security areas were evaluated during Week 1:

- Information Gathering
- Configuration and Deployment Management Testing
- Authentication Testing
- Session Management Testing
- Input Validation Testing
- Error Handling and Information Leakage
- Security Headers and Configuration

Detailed vulnerability results will be documented in the **Findings Details** section of this report.

1.4 Summary of Business Risks

If vulnerabilities exist within the application, attackers may be able to:

- Access sensitive user data
- Perform unauthorized actions
- Exploit authentication or session weaknesses
- Launch client-side attacks such as Cross-Site Scripting (XSS)

- Exploit server misconfigurations
- Gather internal information that can aid further attacks

Such vulnerabilities could lead to **data breaches, reputational damage, financial loss, or service disruption**.

1.5 High-Level Recommendations

Based on initial testing, organizations should generally ensure the following security best practices:

- Implement secure input validation and output encoding.
- Apply strict authentication and authorization controls.
- Configure proper HTTP security headers.
- Avoid information leakage through error messages.
- Perform regular security assessments and vulnerability scans.
- Follow secure development practices aligned with OWASP standards.

Methodology & Tools (OWASP WSTG Based)

3. Automated & Manual Enumeration

3.1 Web Server Identification

Command executed:

```
curl -I https://ae.nothing.tech | grep -i server
```

Result:

```
server: cloudflare server-timing: cfRequestDuration server-timing: ipv
6
```

Analysis

The response headers indicate that the application is deployed behind **Cloudflare CDN/WAF**.

Observed technologies:

Component	Observation
CDN/WAF	Cloudflare
Backend Platform	Shopify
Framework	Hydrogen / Oxygen
Protocol	HTTP/2

Evidence header:

```
powered-by: Shopify, Oxygen, Hydrogen
```

Security Implication

Because Cloudflare sits in front of the origin server:

- The origin server IP is masked.
- Some direct vulnerability testing may be filtered.
- Additional enumeration techniques may be required.

Examples:

- DNS history
- Subdomain enumeration
- Misconfigured endpoints

3.2 HTTP Security Headers Analysis

Command executed:

```
curl -I https://ae.nothing.tech
```

Key Security Headers Identified

Header	Status	Description
Strict-Transport-Security	Present	Enforces HTTPS
Content-Security-Policy	Present	Protects against XSS
X-XSS-Protection	Present	Browser XSS protection
X-Content-Type-Options	Present	Prevents MIME sniffing
X-Permitted-Cross-Domain-Policies	Present	Restricts cross-domain policies

Example header evidence:

```
strict-transport-security: max-age=31536000 x-xss-protection: 1; mode=
block x-content-type-options: nosniff
```

Content Security Policy (CSP)

Observed CSP configuration:

```
content-security-policy: default-src 'self' ...
```

This policy restricts the loading of scripts and resources to trusted domains such as:

- Google Tag Manager
- TikTok Analytics

- Shopify CDN
- Zendesk
- Klaviyo
- Facebook
- Hotjar

Security Assessment

CSP is properly implemented and reduces the risk of:

- Cross-Site Scripting
- Injection of malicious external scripts

However:

```
style-src 'unsafe-inline'
```

Allowing inline styles slightly weakens CSP protection.

Severity: **Low**

3.3 Cookie Security Analysis

Observed cookies:

```
set-cookie: session=...; HttpOnly; Secure; SameSite=None set-cookie: c
ountry_code=ae
```

Cookie Security Attributes

Cookie	Secure	HttpOnly	SameSite
session	Yes	Yes	None
country_code	Not confirmed	No	Not defined

Security Analysis

The session cookie includes appropriate protections:

- Secure flag enabled
- HttpOnly enabled
- HTTPS enforced

However:

```
SameSite=None
```

This configuration allows cross-site requests, which may increase the risk of:

- CSRF attacks

Severity: **Low**

3.4 Directory Enumeration

Tool used:

```
gobuster
```

Command executed:

```
gobuster dir -u https://ae.nothing.tech -w /usr/share/wordlists/dirb/c
ommon.txt
```

Discovered Endpoints

Endpoint	Status	Description
/account	302	Redirects to login
/admin	301	Redirects to Shopify admin
/cart	200	Shopping cart
/offers	200	Promotional page
/policies	200	Policy information
/products	302	Redirects to collections
/search	302	Search page
/support	302	Support center

Example finding:

```
admin (Status: 301) --> https://nothing-ae.myshopify.com/admin
```

Security Observation

The `/admin` endpoint redirects to the Shopify administration panel.

Because Shopify manages authentication and infrastructure, direct exploitation is unlikely unless:

- weak admin credentials
- misconfigured access controls

Severity: **Informational**

3.5 robots.txt Analysis

Command executed:

```
curl https://ae.nothing.tech/robots.txt
```

The file contents revealed restricted directories.

Sensitive Paths Identified

```
/admin /cart /orders /checkouts /carts /recommendations /search
```

Security Analysis

robots.txt exposes internal application paths such as:

- checkout processes
- order endpoints
- cart management

These endpoints may be useful targets during testing, for example:

- parameter manipulation
- business logic flaws
- authentication bypass attempts

Severity: **Informational**

3.6 Sitemap Enumeration

Sitemap reference found:

```
https://ae.nothing.tech/sitemap.xml
```

Sitemap files often expose:

- hidden endpoints

- product URLs
- blog content
- internal pages

This information supports attack surface mapping.

Severity: **Informational**

3.7 Technology Stack Identification

Evidence collected from response headers:

```
powered-by: Shopify, Oxygen, Hydrogen
```

Identified Stack

Component	Technology
E-commerce platform	Shopify
Framework	Hydrogen
Hosting	Shopify Oxygen
CDN/WAF	Cloudflare
Analytics	Google Analytics / TikTok / Facebook
Support	Zendesk

Security Consideration

Since the platform is hosted on Shopify:

- Core infrastructure vulnerabilities are less likely.
- Most vulnerabilities are more likely to occur in:

Examples:

- business logic
- misconfigured endpoints
- client-side scripts
- API interactions

3.8 Attack Surface Summary

Based on enumeration, key accessible areas include:

Area	Endpoint
Authentication	`/account/login`
Shopping Cart	`/cart`
Checkout	`/checkout`
Search	`/search`
Products	`/collections`
Offers	`/offers`

These areas will be the primary focus for:

- input validation testing
- injection testing
- authentication testing

3.9 Preliminary Risk Overview

At this stage, no critical vulnerabilities were identified.

Findings so far:

Severity	Count
Critical	0
High	0
Medium	0
Low	1
Informational	4

---

4. Input Vector Mapping (Burp Suite)

OWASP WSTG Reference: WSTG-INPV-01 – Identify Input Vectors

4.1 Objective

This phase aims to identify all user-controlled input points in the target web application. Input vectors are locations where user data is accepted and processed by the server. These are critical testing points because inadequate validation can lead to vulnerabilities such as:

- Cross-Site Scripting (XSS)
- SQL Injection
- Command Injection
- Insecure Direct Object References (IDOR)
- Business Logic Manipulation

Testing was performed using **Burp Suite Community Edition** as an intercepting proxy to capture and analyze HTTP requests.

4.2 Proxy Configuration

Burp Suite was configured as a local proxy to intercept HTTP and HTTPS traffic from the browser.

Default proxy configuration used:

Setting	Value
Proxy Host	127.0.0.1
Proxy Port	8080
Browser	Firefox
Interception	Enabled

Steps performed:

1. Burp Suite was launched.
2. The proxy listener was confirmed active on **127.0.0.1:8080**.
3. The browser proxy settings were configured to route traffic through Burp Suite.
4. The HTTPS interception certificate was installed to allow inspection of encrypted traffic.

This configuration provided full visibility into all HTTP requests sent to the application.

4.3 Intercepted Application Requests

During manual browsing, multiple HTTP requests were intercepted and analyzed. The following endpoints were identified as accepting user interaction.

Endpoint	Function
`/account/login`	User authentication
`/search`	Product search
`/cart`	Shopping cart
`/collections/shop-all`	Product listings
`/pages/support-centre`	Customer support

These endpoints represent potential locations where user-controlled input may be processed by the application.

4.4 Authentication Endpoint Analysis

Testing focused on the login endpoint:

```
/account/login
```

This page allows users to submit authentication credentials.

Expected parameters:

Parameter	Purpose
email	User email address
password	User password

These parameters are common attack surfaces for:

- SQL Injection
- Authentication bypass
- Credential brute-force attempts

Further testing will be performed in later phases using Burp Repeater and Intruder.

4.5 Search Functionality Input Testing

The search functionality is a user-controlled input point.

Endpoint:

```
/search?q=
```

User-supplied search queries are reflected in the application response and may become potential vectors for:

- Reflected XSS
- Stored XSS
- Injection attacks

Search fields were identified as a high-priority testing area during the assessment.

4.6 Cookie Analysis Through Burp

Burp Suite was used to inspect cookies transmitted between the client and server.

Observed cookies include:

Cookie	Description
session	Application session identifier
country_code	Geographic region identifier

Security attributes observed:

Attribute	Status
Secure	Enabled
HttpOnly	Enabled
SameSite	None

The `Secure` and `HttpOnly` attributes help protect session cookies against:

- Man-in-the-middle attacks
- Client-side script access

However, `SameSite=None` may allow cookies to be sent with cross-site requests.

4.7 Attack Surface Expansion via Burp

Analysis of intercepted traffic identified several external services interacting with the application.

Examples include:

- Google Analytics
- TikTok Analytics
- Facebook tracking
- Zendesk support
- Klaviyo marketing platform

While these integrations are common for e-commerce platforms, they increase the attack surface by introducing additional external dependencies.

Testing will focus only on **in-scope assets** as defined by the engagement rules.

4.8 Initial Testing Preparation

The following Burp Suite modules will be used in upcoming testing phases:

Tool	Purpose
Proxy	Capture and inspect HTTP requests
Repeater	Modify and resend requests
Intruder	Automated payload injection
Decoder	Encode and decode parameters
Comparer	Compare responses

These tools will assist in identifying vulnerabilities in user input handling.

4.9 Summary

The input vector mapping phase identified multiple endpoints where user-supplied input is accepted by the application.

Key input points include:

- Login authentication
- Search queries
- Shopping cart actions
- Product interactions

These endpoints will be further analyzed during the vulnerability testing phase using Burp Suite.

4.10 HTTP Request Analysis (Burp Suite)

Objective

Analyze intercepted HTTP requests and responses to understand how the application processes client requests, cookies, authentication flows, and potential security weaknesses.

Tool Used

- Burp Suite Community Edition
- Browser proxy configured to `127.0.0.1:8080`

Procedure

1. Enabled **Burp Suite Intercept**.
2. Configured the browser proxy to route traffic through Burp.
3. Visited the target application:

```
https://ae.nothing.tech
```

1. Captured multiple HTTP requests during browsing.
2. Analyzed request headers, cookies, authentication flows, and third-party interactions.

Intercepted Request 1 – Chrome Sync Request (Background Traffic)

Request

```
POST /chrome-sync/command/?client=Google+Chrome HTTP/2 Host: clients4.
google.com Authorization: Bearer ya29.a0AT... Content-Type: applicatio
n/octet-stream User-Agent: Chrome LINUX 145.0
```

Analysis

This request was generated by the **Chrome browser synchronization service**, not by the target application.

Observations:

- Request sent to:

```
clients4.google.com
```

- Uses **Bearer Token authentication**
- Payload is compressed using **gzip**
- Contains browser sync metadata

Security Assessment

Parameter	Result
Target Domain	External
In Scope	No
Security Impact	None

Conclusion

This request is **background browser traffic** and is unrelated to the target application. According to the testing scope:

```
In Scope: https://ae.nothing.tech Out of Scope: https://*.nothing.tech
External domains
```

Therefore, this request was **ignored for vulnerability analysis**.

Intercepted Request 2 – Authentication Redirect

Request

```
GET /login/?source=store&store=nothing-ae HTTP/2 Host: account.nothin
g.tech
```

Important Parameters

Parameter	Value
source	store
store	nothing-ae
callback	https://ae.nothing.tech/account/login
return_to	https://ae.nothing.tech/account

Cookie Data

Multiple cookies were observed:

```
country_code=ae _ga _fbp _shopify_y _shopify_s rl_session ttcsid
```

Observations

1. Authentication is handled by a **separate domain**:

```
account.nothing.tech
```

1. The application uses **OAuth-style redirection** with the following parameters:

```
callback return_to
```

1. Session tracking cookies are present.

Security Testing Performed

The following checks were conducted:

Test	Result
Parameter Tampering	No change in behavior
Open Redirect Test	Not vulnerable
Cookie Manipulation	No privilege escalation

Example Test Payload

Modified request:

```
return_to=https://google.com
```

Result:

```
Application ignored external redirect
```

Security Assessment

Vulnerability	Status
Open Redirect	Not Found
Authentication Bypass	Not Found
Parameter Tampering	Not Found

Intercepted Request 3 – Main Website Request

Request

```
GET / HTTP/2 Host: ae.nothing.tech
```

Cookies Observed

```
session=e30... country_code=ae _shopify_y _shopify_s _ga _fbp _uetsid
```

Observations

1. The site uses **Shopify platform infrastructure**.
2. A **session cookie** is present:

```
session=e30...
```

1. Analytics and tracking cookies were detected:

- Google Analytics
- Facebook Pixel
- Microsoft Clarity
- TikTok tracking

Security Analysis

Test	Result
Session Cookie Manipulation	No change
Cookie Deletion	Session recreated
Session Hijacking Simulation	Not possible without a valid session

Cookie Security Review

Cookie	Secure Flag	HttpOnly	SameSite
session	Enabled	Enabled	Lax
_shopify_y	Enabled	Not HttpOnly	Lax
_shopify_s	Enabled	Not HttpOnly	Lax

Findings

No critical vulnerabilities were identified during request analysis. However, the presence of multiple third-party tracking cookies may increase the **attack surface for tracking or privacy concerns**.

Summary

Step	Description	Result
3.7	Directory discovery	No sensitive directories
3.8	Parameter analysis	No vulnerabilities
3.9	Burp interception	Traffic captured
3.10	HTTP request analysis	No exploitable issues

Authentication Testing

OWASP WSTG-ATHN

Objective

Assess the application's authentication mechanism to identify weaknesses such as authentication bypass, weak login controls, credential exposure, and improper session handling.

Target

```
https://ae.nothing.tech
```

Authentication service observed during testing:

```
https://account.nothing.tech
```

This indicates that authentication is handled by a **centralized account service**.

4.1 Authentication Entry Points Discovery

Objective

Identify all authentication entry points in the application.

Methodology

The following steps were performed:

1. Navigated the application manually.
2. Identified login links on the website.
3. Observed redirection behavior.
4. Intercepted requests using **Burp Suite Proxy**.

Identified Login Endpoint

When selecting **Login / Account**, the browser was redirected to:

```
https://account.nothing.tech/login
```

Captured Login Request

```
GET /login/?source=store&store=nothing-ae&callback=https://ae.nothing.
tech/account/login&return_to=https://ae.nothing.tech/account HTTP/2 Ho
st: account.nothing.tech
```

Parameters Observed

Parameter	Description
source	Source of the authentication request
store	Store identifier
callback	URL where the authentication response returns
return_to	Final redirect destination

Security Testing

The following tests were conducted:

Test	Result
Parameter manipulation	No effect
Invalid callback URL	Request rejected
External redirect attempt	Not allowed

Example Test

Modified parameter:

```
return_to=https://evil.com
```

Result:

```
Application ignored external redirect
```

Result

No **open redirect vulnerability** was detected.

4.2 Authentication Mechanism Identification

Objective

Identify the authentication mechanism used by the application.

Observations

During testing, the following was observed:

1. Authentication occurs on:

```
account.nothing.tech
```

1. The main store application:

```
ae.nothing.tech
```

redirects users to the authentication domain.

1. This indicates a **centralized authentication architecture**.

Authentication Flow

1. User selects **Login**.
2. User is redirected to:

```
account.nothing.tech/login
```

1. User authenticates.
2. An authentication token is returned.
3. User is redirected back to:

```
ae.nothing.tech/account
```

Security Considerations

Security Feature	Status
HTTPS enforced	Yes
Secure cookie usage	Observed
External redirect prevention	Observed

Result

The authentication system appears to be securely implemented, with proper redirect handling.

4.3 Login Page Security Testing

Objective

Analyze the login page for security weaknesses.

Testing Method

The login page was analyzed using:

- Manual inspection
- Burp Suite interception
- Source code analysis

Checks Performed

Test	Result
Password field masking	Enabled
HTTPS login submission	Enabled
Credential exposure in URL	Not detected
JavaScript sensitive data leakage	Not detected

Input Fields Observed

Typical login inputs include:

```
Email Address Password Login Button
```

Result

The login page does not expose credentials in the URL and enforces secure HTTPS communication.

4.4 Authentication Rate Limiting Testing

Objective

Test whether the application prevents **brute-force login attacks**.

Methodology

The following tests were conducted:

1. Attempted multiple logins using invalid credentials.
2. Observed response behavior.
3. Checked for account lockout or CAPTCHA.

Test Scenario

```
Tested 5+ invalid login attempts
```

Observations

Behavior	Result
Account lockout	Not observed
CAPTCHA triggered	Not observed
Error message returned	Yes

Example Response

```
Invalid email or password
```

Security Risk

If rate limiting is not in place, attackers may perform **credential brute-force attacks**.

Severity

```
Low Risk
```

Recommendation

Implement the following protections:

- Account lockout after multiple failed attempts
- CAPTCHA verification
- Login rate limiting
- IP-based request throttling

Step 4 Progress

Step	Status
4.1 Authentication Entry Points	Completed
4.2 Authentication Mechanism	Completed
4.3 Login Page Analysis	Completed
4.4 Rate Limiting Test	Completed

4.5 CAPTCHA Protection Testing

OWASP WSTG-ATHN-03

Objective

Evaluate whether the application implements CAPTCHA protection to deter automated login attempts and brute-force attacks.

Tools Used

- Burp Suite Proxy
- Browser Developer Tools

Procedure

1. Navigated to the login page:

```
https://account.nothing.tech
```

1. Intercepted authentication traffic using **Burp Suite**.
2. Observed network requests generated during the login flow.
3. Identified CAPTCHA-related verification requests.

Captured CAPTCHA Request

```
POST /recaptcha/api2/clr?k=6LexIo0gAAAAAKY_t7-nBmRm1ADHkB7QjYXYyTn_ HTTP/2 Host: www.recaptcha.net Origin: https://account.nothing.tech Referer: https://account.nothing.tech/ User-Agent: Mozilla/5.0 (X11; Linux x86_64) Chrome/145.0.0.0
```

Request Description

This request is generated by **Google reCAPTCHA** when the challenge is loaded or validated.

Important Fields

Field	Description
POST	HTTP method used for CAPTCHA verification
Host	`www.recaptcha.net`
Origin	`account.nothing.tech`
Referer	Login page
k parameter	reCAPTCHA site key

Observations

The application integrates **Google reCAPTCHA** as an anti-automation control.

```
k=6LexIo0gAAAAAKY_t7-nBmRm1ADHkB7QjYXYyTn_
```

This key represents the **site-specific CAPTCHA configuration**.

Security Behavior

During authentication testing, the following behaviors were observed:

Test	Result
CAPTCHA triggered on login page	Yes
CAPTCHA request sent to Google	Yes
Automated login protection	Enabled
CAPTCHA bypass attempt	Not successful

Security Analysis

Google reCAPTCHA helps reduce the risk of:

- Automated login attacks
- Credential stuffing
- Bot traffic
- Brute-force password guessing

Risk Evaluation

Risk	Status
CAPTCHA Missing	No